Softwareentwurf

Joern Ploennigs · AI4SC



> Simplicity is prerequisite for reliability.
>
> — Edsger W. Dijkstra

Von der Anforderung zum Code

Bevor eine einzige Zeile Code geschrieben wird, muss klar sein, *was* das Programm leisten soll und *wie* es strukturiert werden soll. Dieser Abschnitt zeigt, wie die Lücke zwischen einer Anforderung des Auftraggebers und einer Zeile Python geschlossen wird — und warum dieser Entwurfsschritt entscheidend ist.

Die Lücke: Anforderung → Code

Zwischen der Aussage eines Auftraggebers und ausführbarem Python-Code liegt eine fachliche Übersetzungsleistung. Menschen beschreiben meist Ziele in ihrer Domänensprache, etwa „Wasserverbrauch pro Raum berechnen“. Entwickler müssen daraus jedoch ein präzises Modell ableiten: Welche Objekte gibt es, welche Daten speichern sie, welche Operationen müssen sie anbieten und wie hängen sie zusammen?

Genau diese Lücke ist der Kern des Softwareentwurfs. Ohne diesen Zwischenschritt wird Code schnell zu einer Ansammlung von Einzelentscheidungen, die zwar lokal funktionieren, aber kein tragfähiges Gesamtsystem ergeben.

Entwurf schließt die Lücke

Der Entwurf ist die Brücke zwischen Anforderung und Implementierung. Er macht aus einer unscharfen Beschreibung ein prüfbares Modell, das sowohl im Team als auch mit KI-Werkzeugen kommuniziert werden kann. In diesem Kurs ist das meist ein UML-Klassendiagramm (→ `04c_Objects`).

Erst wenn dieses Modell steht, wird implementiert. Der große Vorteil: Fehler im Verständnis werden früh sichtbar, noch bevor Zeit in Code investiert wird. Deshalb gehört zum Entwurf nicht nur das Zeichnen von Klassen, sondern auch die Frage, wie der spätere Code gegen das Modell verifiziert wird.

Softwareentwurf als Projektplan

Software wird selten allein entwickelt, sondern im Team über einen längeren Zeitraum. Dafür braucht es einen Projektplan, der beschreibt, wie die Arbeit aufgeteilt wird. Dieses Erstellen des Projektplans heißt Softwareentwurf und ähnelt stark dem Entwurfsvorgehen im Umwelt- und Bauingenieurwesen.

Der Projektplan besteht aus zwei Teilen: dem Klassenentwurf, der dem Bauplan entspricht, und dem Programmiervorgehen, das der Bauablaufplanung entspricht.

Die Phasen der Softwareentwicklung

Anforderungsdefinition

Die erste Phase dient der präzisen Definition dessen, was die Software leisten soll. Hier werden sowohl die funktionalen Aspekte als auch die Randbedingungen festgelegt. Im Bauingenieurwesen entspricht dies der Ausschreibung, in der dokumentiert wird, welche Funktionen und Randbedingungen das Bauwerk erfüllen muss.

Bei einer Brücke werden beispielsweise die zulässigen Verkehrslasten, die zu überbrückende Spannweite und die geforderte Lebensdauer definiert. Für eine Kläranlage werden die Anzahl der anzuschließenden Einwohner, der geforderte Reinigungsgrad und die einzuhaltenden Grenzwerte spezifiziert. Genauso müssen bei der Softwareentwicklung alle funktionalen Anforderungen, Leistungsparameter und technischen Beschränkungen klar formuliert werden.

Entwurfsphase

Der Entwurf definiert, wie die Software strukturell aufgebaut werden soll. Diese Phase gliedert sich in die Grobplanung, bei der die Software in logische Module aufgeteilt wird, und die Feinplanung, die festlegt, wann welche Komponenten von wem entwickelt werden.

Im Bauwesen entspricht dies der Architektur- und Entwurfsplanung. Bei einem Hochbau wird zunächst die Grundstruktur mit Fundament, Rohbau und Ausbau geplant, bevor die Details wie Bewehrungspläne und Installationen ausgearbeitet werden. Ähnlich wird bei der Softwareentwicklung erst die Gesamtarchitektur entworfen, bevor die einzelnen Klassen und ihre Schnittstellen definiert werden.

Ausführungsphase

Die Ausführung umfasst sowohl die Implementation der einzelnen Softwarekomponenten als auch deren Integration zu einem funktionsfähigen Gesamtsystem. Während der Implementation werden die einzelnen Klassen und Module nach den Vorgaben des Entwurfs programmiert. Die anschließende Integration führt diese Einzelteile zur fertigen Lösung zusammen.

Im Bauwesen arbeiten verschiedene Gewerke parallel an ihren spezifischen Aufgaben, bevor ihre Arbeiten koordiniert zusammengeführt werden. Bei einem Wasserbauprojekt werden beispielsweise Pumpstationen, Rohrleitungen und Steuerungssysteme zunächst separat erstellt und dann zu einem funktionsfähigen Wasserversorgungssystem verbunden.

Abnahme und Test
Die Abnahmephase stellt sicher, dass die fertige Software alle Anforderungen erfüllt. Dies geschieht in mehreren Stufen: Zunächst werden einzelne Module isoliert getestet, dann wird das Zusammenspiel zwischen den Modulen geprüft, und schließlich wird das Gesamtsystem unter realen Bedingungen getestet.

Diese Vorgehensweise entspricht der Bauabnahme, bei der ebenfalls stufenweise vorgegangen wird. Bei einer Kläranlage werden zunächst einzelne Pumpen auf ihre Funktion geprüft, dann das Zusammenspiel von Steuerung, Pumpen und Sensoren getestet, und schließlich wird die gesamte Anlage unter Realbedingungen abgenommen.

Vergleich: Software- vs. Bauingenieurwesen
Der Vergleich mit dem Bauingenieurwesen ist hilfreich, weil dort die Trennung zwischen Anforderung, Entwurf und Ausführung selbstverständlich ist. Niemand beginnt mit dem Gießen des Fundaments, bevor Entwurfszeichnungen, Fachplanung und Freigaben vorliegen.

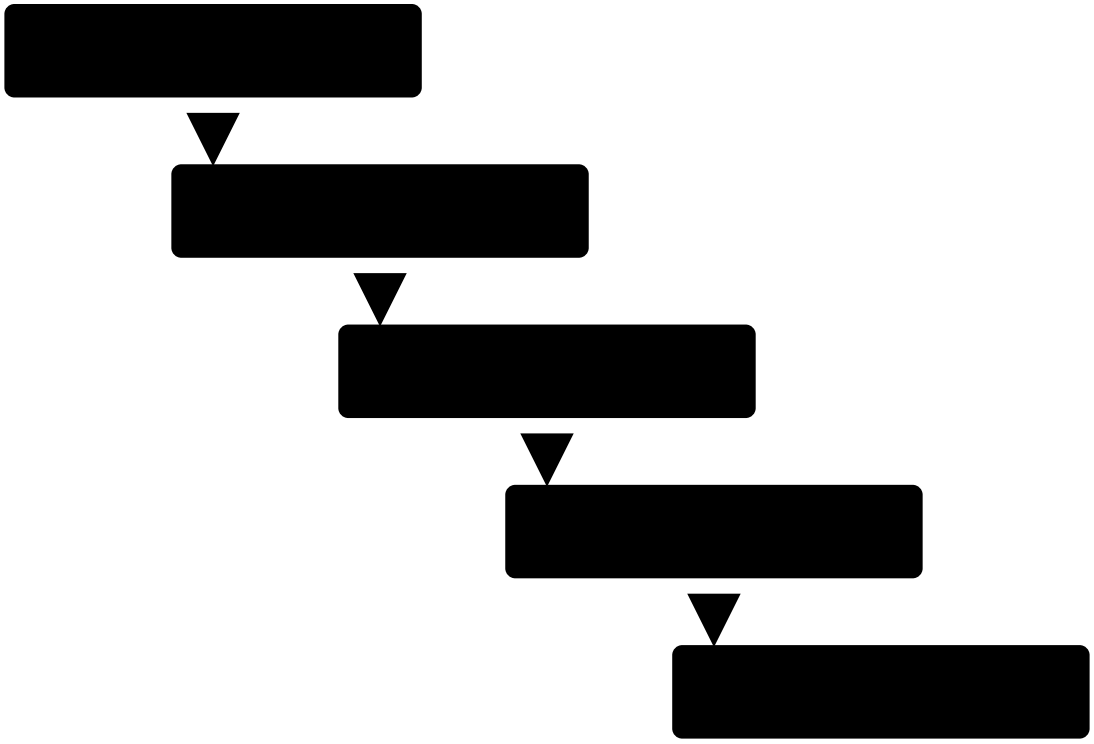
In der Softwareentwicklung ist die Versuchung größer, sofort mit dem Codieren zu beginnen, weil Änderungen scheinbar billig sind. Gerade deshalb ist der Entwurf wichtig: Er erfüllt dieselbe Funktion wie ein Plan im Bauwesen, nur dass hier nicht Beton und Stahl, sondern Klassen, Datenflüsse und Schnittstellen koordiniert werden.

Vorgehensmodelle
Software wird heute fast ausschließlich im Team über längere Zeiträume entwickelt. Unterschiedliche Projekttypen erfordern unterschiedliche Vorgehensmodelle — das richtige Modell zu wählen ist selbst eine Entwurfsentscheidung.

Welches Vorgehensmodell passt?
Die Wahl des Vorgehensmodells hängt vom Projekt ab. Wasserfall passt zu stabilen Anforderungen und formalen Ausschreibungen, prüft aber erst am Ende und entdeckt Fehler früher Phasen spät. Das V-Modell definiert die Tests schon im Entwicklungsarm und eignet sich für sicherheitskritische Software, ist dafür aufwendiger. Agile Entwicklung reagiert auf sich ändernde Anforderungen, prüft in jeder Iteration, riskiert aber einen Neuentwurf bei frühen Entwurfsfehlern. Beim agentischen Loop erzeugt eine KI den Code, und der Mensch verifiziert ihn nach jeder Generierung.

Wasserfallmethode - Das traditionelle Entwicklungsmodell
Die Wasserfallmethode repräsentiert den traditionellen Ansatz der Softwareentwicklung und wird noch heute häufig in Ausschreibungen großer Systeme gefordert. Ihr charakteristisches Merkmal ist die streng sequenzielle Abarbeitung der Entwicklungsphasen, wobei jede Phase vollständig abgeschlossen werden muss, bevor die nächste beginnt.

Diese Methode zeichnet sich durch eine begrenzte Nutzerbeteiligung aus, die hauptsächlich auf die Anforderungsdefinition beschränkt ist. Jede Aktivität wird umfassend dokumentiert, was die Methode besonders für Ausschreibungen und ISO-9000-konforme Entwicklungen geeignet macht.



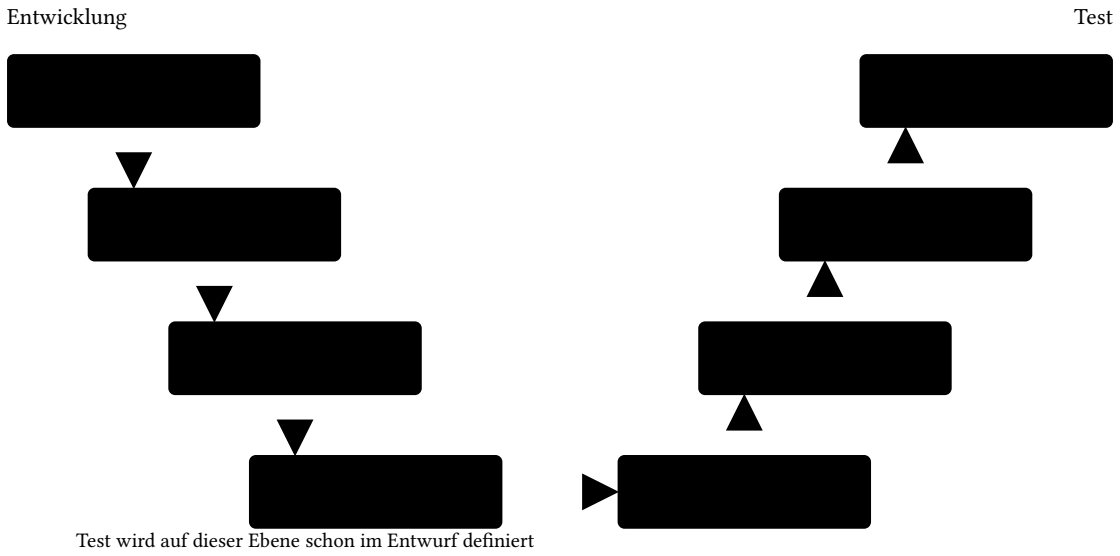
Der große Vorteil dieser Methode liegt in ihrer Planbarkeit und dem strukturierten Vorgehen. Kosten und Termine lassen sich präzise schätzen, und die umfassende Dokumentation erfüllt alle formalen Anforderungen. Allerdings führt die sequenzielle Natur dazu, dass Fehler aus frühen Phasen erst sehr spät entdeckt werden, was teure Nachbesserungen zur Folge haben kann.

Die Wasserfallmethode entspricht dem traditionellen Bauvorgehen. Bei einer Autobahnbrücke werden zunächst alle Anforderungen in der Ausschreibung definiert, dann erfolgt die komplette Planung von der Statik bis zur Ausführungsplanung, bevor mit dem eigentlichen Bau begonnen wird. Erst nach Fertigstellung des gesamten Bauwerks erfolgen die Belastungstests und die Abnahme.

V-Methode - Entwicklung mit systematischer Testplanung

Die V-Methode wurde speziell für die Entwicklung sicherheitskritischer Software konzipiert und findet hauptsächlich in der Automobil- und Luftfahrtindustrie Anwendung. Sie erweitert das Wasserfallmodell um eine systematische Testplanung, die bereits während der Entwicklungsphase beginnt.

Das charakteristische V-förmige Modell trennt Entwicklungs- und Testaktivitäten klar voneinander. Während auf der linken Seite die eigentliche Entwicklung stattfindet, werden auf der rechten Seite die entsprechenden Tests definiert und durchgeführt. Jede Entwicklungsphase hat ihre korrespondierende Testphase, wodurch eine hohe Testabdeckung erreicht wird.



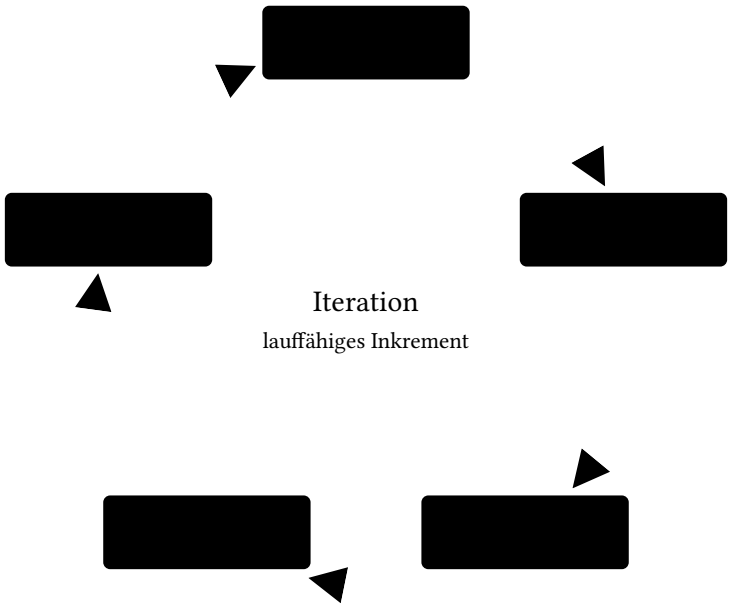
Die V-Methode bietet den Vorteil einer systematischen Qualitätssicherung mit hoher Rückverfolgbarkeit. Allerdings ist sie aufwendiger als andere Methoden und eignet sich daher primär für Bereiche, in denen Sicherheit und Zuverlässigkeit oberste Priorität haben.

Im Tunnelbau, einem sicherheitskritischen Bereich des Bauingenieurwesens, wird ähnlich vorgegangen. Bereits während der Planung der Sicherheitssysteme werden die entsprechenden Testverfahren definiert. Für jeden Brandmelder wird festgelegt, wie er getestet wird, für das Zusammenspiel von Brandmeldung und Lüftungssteuerung werden Integrationstests geplant, und für das Gesamtsystem wird eine komplette Evakuierungsübung mit der Feuerwehr konzipiert.

Agile Methode - Flexibilität durch iterative Entwicklung

Die agile Methode stellt eine moderne Antwort auf die Herausforderungen sich schnell ändernder Anforderungen dar. Anstatt ein komplettes System von Anfang bis Ende zu planen, wird schrittweise entwickelt und kontinuierlich verbessert.

Der agile Ansatz beginnt mit einem Minimal Viable Product, einer einfachsten funktionsfähigen Version der Software. Diese wird dann in regelmäßigen Zyklen erweitert und verbessert, wobei jede Iteration neue Funktionen hinzufügt oder bestehende optimiert. Die Entwicklungszyklen sind typischerweise kurz und dauern meist nur wenige Monate.



Der große Vorteil der agilen Methode liegt in ihrer Flexibilität und der Möglichkeit, frühzeitig Feedback zu erhalten. Benutzer können bereits funktionsfähige Versionen testen und Verbesserungsvorschläge einbringen. Dies reduziert das Risiko, an den tatsächlichen Bedürfnissen vorbei zu entwickeln.

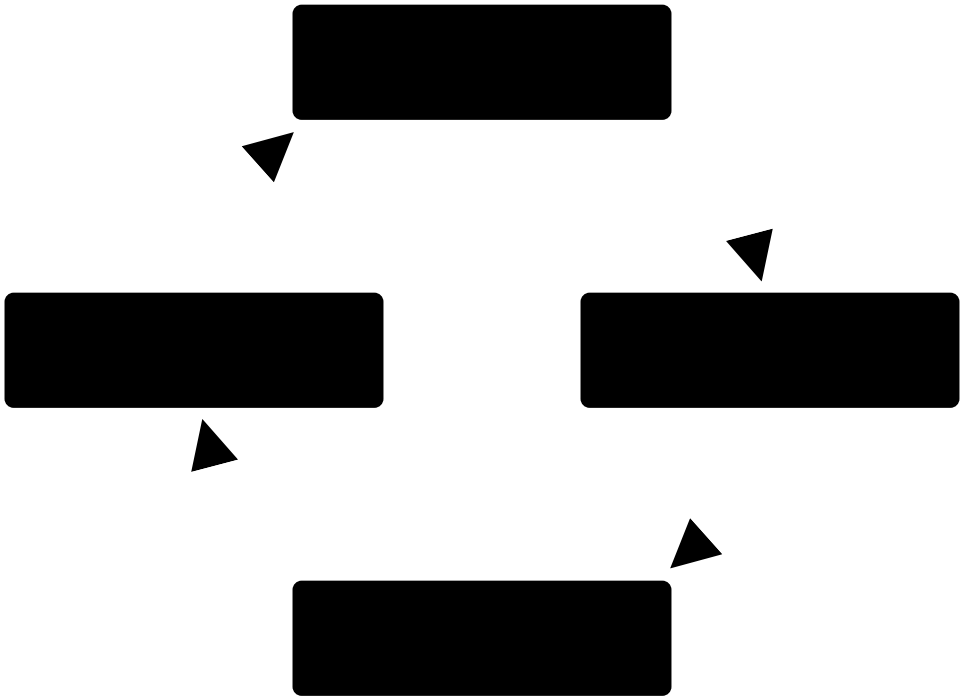
Allerdings birgt die agile Methode auch Risiken. Entwurfsfehler in frühen Iterationen können dazu führen, dass die gesamte Architektur überarbeitet werden muss. Zudem ist die Kosten- und Zeitplanung schwieriger, da das Endprodukt zu Projektbeginn nicht vollständig spezifiziert ist.

Ein Beispiel aus dem Bauwesen ist das modulare Bauen, was insbesondere im Holzbau zunehmend beliebter wird. Anstatt eine Bauwerk komplett zu planen und zu bauen, werden dort zuerst einzelne Module geplant, die man immer wieder miteinander kombinieren kann. Die Module kann man einzeln aufbauen und testen, bevor man sie im finalen Bauwerk zusammen setzt.

Agentischer Entwicklungs-Loop — das vierte Modell

Der agentische Entwicklungs-Loop ist ein modernes viertes Vorgehensmodell, das die Verfügbarkeit von KI-Coding-Assistenten voraussetzt. Er folgt dem gleichen iterativen Rhythmus wie die agile Methode, aber die KI übernimmt die Implementierungsphase.

> **i** Hinweis
>
> Diese Zuordnung ist ein aktueller Praxistrend im Umgang mit KI-Werkzeugen und noch kein klassisches, über Jahrzehnte standardisiertes Vorgehensmodell wie Wasserfall, V-Modell oder Scrum.



Vier Vorgehensmodelle im Vergleich

Die vier Modelle beantworten dieselbe Grundfrage auf unterschiedliche Weise: Wann wird geplant, wann wird implementiert und wann wird geprüft? Wasserfall priorisiert Planbarkeit, das V-Modell Nachweisbarkeit und Tests, agile Entwicklung schnelle Rückkopplung bei unsicheren Anforderungen, und der agentische Loop verschiebt den Schwerpunkt zusätzlich auf das Verstehen und Verifizieren von KI-generiertem Code.

Ein einfaches Auswahlbeispiel: Angenommen, ein Team entwickelt die Steuerungssoftware für eine Tunnelanlage. Die Anforderungen sind weitgehend bekannt, Ausfälle wären sicherheitskritisch, und jede Funktion muss nachvollziehbar geprüft werden. Hier passt das *V-Modell* besser als eine rein agile Vorgehensweise, weil Anforderungen, Tests und Implementierung systematisch aufeinander abgebildet werden können.

Anders wäre es bei einem jungen Start-up, das eine neue Gebäudeverwaltungs-App entwickelt und noch nicht weiß, welche Funktionen Nutzer wirklich brauchen. Dort ist ein *agiles* Vorgehen meist sinnvoller, weil kurze Iterationen schnelles Feedback ermöglichen.

Damit löst sich auch die Einstiegsfrage „Was entsteht zuerst — Code oder Plan?“: In moderner Softwareentwicklung entsteht zuerst ein *Plan*, aber dieser Plan ist kein starres Dokument. Er ist ein präzises Modell, aus dem Code erzeugt, diskutiert und verifiziert werden kann.

Bibliographie